

以容器化技術建構分散式 TSP 路徑優化平台

專題提案書

提案主題：物流配送路徑最佳化與分散式運算

系統形式：Total 4 Containers (1 Master + 3 Workers)

預期成果：以三個計算節點並行縮短 TSP 求解時間，並具備節點監控與容錯能力

文件版本：v1.0

日期：2026 年 5 月 14 日

提案摘要

本專題聚焦於物流業常見的旅行推銷員問題 (Traveling Salesman Problem, TSP)。傳統單機求解方式在配送點數量增加時，容易面臨運算時間急遽上升的問題，不利於即時或近即時的配送規劃。為此，本提案規劃以容器化技術建立分散式運算平台，由 Master Container 統一接收任務、分派工作並整合結果，三個 Worker Container 則並行執行路徑優化演算法，藉由水平分工降低單一節點負載，縮短整體求解時間，並提升系統穩定度與可擴充性。

本提案的核心價值在於：

- 將 TSP 求解任務切分後並行處理，降低等待時間。
- 透過容器化部署，讓每個計算節點具備一致環境，便於維護與擴充。
- 以視覺化監控介面管理任務狀態、節點資源與執行結果。
- 加入 heartbeat 與工作重分配機制，形成具基本高可用性的分散式架構。

一、專題背景與問題定義

物流配送常需要在多個配送點之間找出成本最低、路徑最短或時間最少的拜訪順序。TSP 雖然是經典最佳化問題，但其計算複雜度會隨配送點數量快速增加，當實務情境中需要在短時間內處理大量訂單、臨時改單或即時路況變化時，單機求解的延遲往往成為瓶頸。

本提案的問題意識，不只是要「找到一條最短路徑」，而是要在可接受時間內提供可用解，並能在任一節點出現異常時，維持整體服務不中斷。因此，本系統以容器化與分散式運算為主軸，讓求解任務可被切分、追蹤、監控與重派，使最佳化能力能真正落地到物流作業情境。

二、專題目標

- 建立一套可部署於單機或小型伺服器上的容器化分散式運算平台。
- 透過三個 Worker 並行運算，縮短 TSP 路徑搜尋與評估時間。
- 提供可視化任務狀態、節點資源使用率與結果匯總畫面。
- 支援任務刪除、錯誤任務排除與重新分派，提升操作便利性。
- 加入基本高可用性設計，降低單點故障造成的服務中斷風險。

三、系統架構設計

整體架構採 Total 4 Containers 設計，包含一個 Master Container 與三個 Worker Containers。Master Container 作為 Web Manager 與 Job Dispatcher，負責接收前端任務、拆分與派送工作、追蹤節點執行狀況，並在任務完成後匯整結果。Worker A、Worker B、Worker C 則各自負責接收座標資料與執行路徑優化演算法。

容器	主要職責	重點輸出
Master	前端介面、任務分配、狀態監控、結果彙整	任務編號、節點指派、最終路徑
Worker A	接收任務並執行 TSP 求解	局部最佳路徑、執行時間

容器	主要職責	重點輸出
Worker B	接收任務並執行 TSP 求解	局部最佳路徑、執行時間
Worker C	接收任務並執行 TSP 求解	局部最佳路徑、執行時間

此架構的優點在於：

- Master 與 Worker 分離，邏輯清楚，便於維護與擴充。
- 三個 Worker 可同時處理不同任務區段，提升整體吞吐量。
- 容器之間環境一致，可降低部署時的版本差異與相依性風險。

四、核心功能實作

4.1 任務狀態監控

系統前端將即時呈現任務狀態，至少包含「排隊中」、「執行中」與「已完成」三種狀態。當任務進入執行階段時，畫面需同步標明由哪一個節點負責處理，讓使用者可以快速辨識目前運算位置與進度。

4.2 資源數據監看

各節點容器將週期性回傳系統資源狀態，並整合 `top -bn 1 -i -c` 指令結果，即時取得 CPU 與記憶體使用率。透過同一畫面統整 Master 與三個 Worker 的資源狀況，可幫助開發與測試階段快速判斷瓶頸是否出現在單一節點。

4.3 任務管理

- 允許使用者在運算開始前，從排隊隊列中刪除錯誤任務。
- 支援刪除重複訂單或已失效的座標資料，避免浪費計算資源。
- 任務刪除後需同步更新前端狀態與後端工作佇列，避免殘留假資料。

4.4 任務處理流程

1. 使用者於前端提交配送座標與任務名稱。

2. Master 驗證資料格式並將任務加入排隊佇列。
3. Dispatcher 依據節點狀態將任務派送至可用 Worker。
4. Worker 執行路徑求解並回傳最佳結果與耗時。
5. Master 匯總結果並將狀態更新為已完成。

五、高可用性與容錯設計

為了讓系統不只是能跑，也能在節點異常時維持服務，本提案規劃三層容錯思維：第一層是 heartbeat 偵測，第二層是工作回收與重分配，第三層是失效節點剔除與群組更新。

5.1 心跳偵測 (Heartbeat)

Master 將定時向 Worker 發送健康檢查請求；若在設定時間內沒有收到回應，即可判定該節點可能發生當機、網路中斷或服務異常。此機制的目的不是追求絕對嚴苛的誤差為零，而是用低成本方式快速偵測節點是否可持續承接任務。

5.2 工作重新分配

- 若 Worker A 執行到一半失去回應，Master 必須保留尚未完成的任務資訊。
- Master 將該任務從失效節點的執行清單中回收，並重新指派給 Worker B 或 Worker C。
- 重新分配時需保留任務上下文與輸入座標，確保結果一致且可追蹤。

5.3 節點剔除

當連續多次 heartbeat 都無法恢復連線時，系統將自動將該失效節點從可用計算群組中移除，避免 Dispatcher 持續向故障容器派發新工作。這個邏輯可降低重試成本，也讓整體排程行為更穩定。

六、開發規劃與里程碑

本專題建議以四個階段推進：需求釐清、系統實作、整合測試與提案展示。由於本案重點在可展示的架構與功能完整性，因此每一階段都應保留可視化成果，例如任務畫面、資源監控畫面、心跳異常畫面與重新分派結果。

階段	工作內容	產出
1	需求與架構設計	任務流程、資料格式、節點角色定義
2	Master / Worker 開發	任務分派、求解模組、狀態回報
3	監控與容錯整合	heartbeat、重新分配、資源監看
4	展示與驗收	可運行 Demo、報告與簡報素材

七、預期效益

- 縮短配送路徑求解時間，提高物流規劃效率。
- 提供清楚的任務狀態與節點資源資訊，提升系統透明度。
- 透過容器化部署，使系統更容易複製、測試與擴充。
- 加入故障偵測與重派機制，提升系統可用性與實務應用價值。

八、風險與對策

風險	對策
任務過大導致單節點執行時間過長	將任務拆分為可管理的工作單元，或以分批派送降低單一節點壓力。
節點故障造成中斷	以 heartbeat 與重分配機制將未完成任務回收並派發至其他 Worker。
監控資訊過多影響介面可讀性	前端只保留必要指標，細節資訊可展開或另設查詢頁面。
容器之間通訊延遲	優先使用輕量 API 設計，並減少不必要的頻繁同步。

九、結語

本提案以物流業實際面臨的 TSP 計算瓶頸為出發點，透過 Master / Worker 的容器化分散式架構，不只提升運算效率，也把任務監控、資源觀測與容錯重分配一併納入設計。若後續能完成實作與

驗證，本系統將可作為小型物流最佳化平台的雛形，並進一步延伸至更多配送情境、更多節點與更複雜的最佳化演算法。

以上為本專題提案內容，敬請指導。